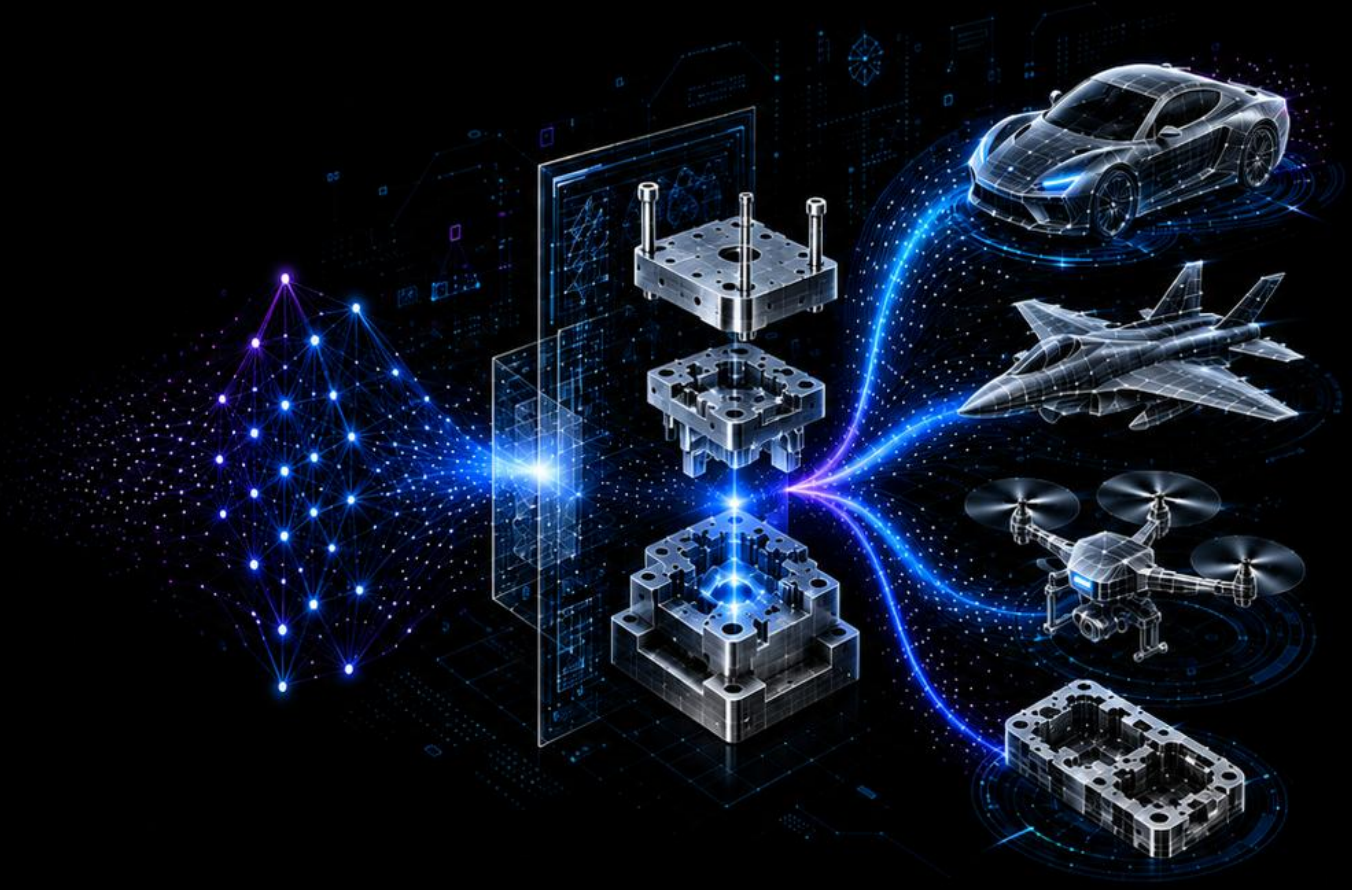
8장 스터디 : 성능 최적화

스터디 보고



정승한
(지도교수: 양선웅)

한양대학교 ERICA 기계공학과

Intelligent Design for Engineering Applications Lab

8.1.1 데이터를 사용한 성능 최적화

데이터 수집하기

- 최대한 많은 데이터를 수집하는 것이 성능 최적화에 가장 좋으나 어려운 경우 임의로 데이터를 생성하는 방법을 고려함
- 일반적으로 딥러닝이나 머신러닝 알고리즘은 데이터 양이 많을수록 성능이 좋음
- 공개 데이터 셋을 활용하는 방법도 존재

데이터 생성하기

- 데이터 증강
- 기존 데이터를 조금씩 변형하여 새로운 데이터처럼 사용하는 방법
- 이미지에서는 회전, 좌우반전, 확대, 위치 이동 등의 변형을 많이 사용
- 시계열 데이터에서는 노이즈 추가, 시간 축 이동, 신호 크기 조절을 주로 사용

데이터 범위 조정하기

- 데이터 범위 조정은 두가지 의미로 생각할 수 있음
- 입력 값의 숫자 범위를 조정 예를 들어 변수의 크기를 비슷하게 맞추는 정규화, 표준화가 존재하고, 학습 조건의 범위를 넓히는 조정이 존재함

알고리즘을 이용한 성능 최적화

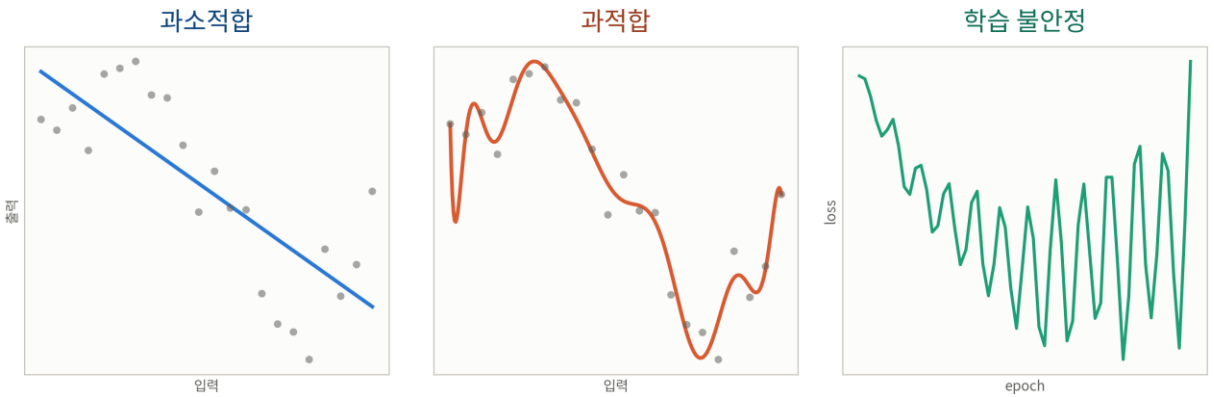
❖ 유사한 용도의 알고리즘들을 선택하여 모델을 훈련시켜 보고 최적의 성능을 보이는 알고리즘을 선택

❖ 분류를 위한 SVM, KNN등의 알고리즘을 이용하여 훈련, 시계열의 경우 RNN, LSTM, GRU 등을 이용하여 훈련하며 가장 좋은 모델을 선택함

알고리즘 튜닝을 위한 최적화

❖ 진단 : Train Loss와 Test Loss가 감소하고 정확도가 증가하면 정상

- 과소적합 : 정확도가 낮고, 학습을 반복해도 성능이 잘 오르지 않음. 네트워크가 너무 단순하거나, 데이터가 부족한 경우 발생
  - ✓ 노드, hidden layer 증가, 에포크 증가, 활성화 함수 변경 등으로 해결 가능
- 과적합 : 모델이 훈련 데이터를 지나치게 외워 새로운 데이터에 제대로 대응하지 못함
  - ✓ Dropout 적용, hidden layer 감소, 데이터 증강으로 해결 가능
- 학습 불안정 : Loss가 크게 진동하거나 증가하는 경우
  - ✓ Learning rate 줄이기, 배치 정규화 적용, Gradient Clipping 적용



가중치

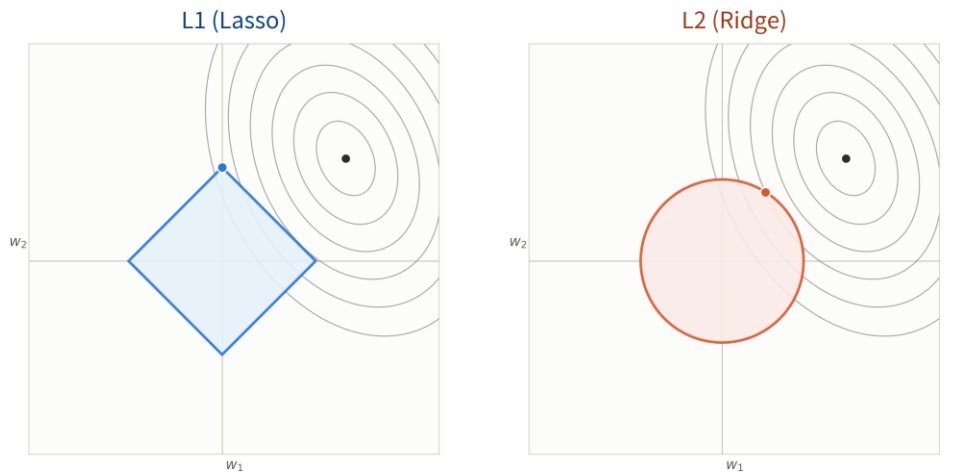
- 가중치는 입력 데이터가 결과에 영향을 얼마나 미치는지를 학습
- 가중치는 처음에 임의의 값으로 초기화되며 Loss function을 줄이는 방향으로 수정
- 가중치 초기화
  - 가중치 학습을 모두 0에서 시작한다면 모든 뉴런들이 똑같은 값을 계산하고 업데이트
  - 작은 무작위 값으로 초기화됨

가중치 초기화

- Xavier 초기화
  - Sigmoid나 Tanh 같은 activation function과 함께 사용
  - $Var(w) = \frac{2}{n_{in}+n_{out}}$
  - 가중치의 분산을 입력 뉴런 수와 출력 뉴런 수에 따라 조절
- He 초기화
  - ReLU나 Leaky ReLU에서 주로 사용
  - $Var(w) = \frac{2}{n_{in}}$

가중치 정규화

- 가중치가 지나치게 커지면 과적합 될 가능성이 존재
- L1 정규화
  - $L_{total} = L_{data} + \lambda \sum_i |w_i|$
- L2 정규화
  - $L_{total} = L_{data} + \lambda \sum_i {w_i}^2$



사용자 지정 변수

❖ Learning rate

- Rate가 너무 큰 경우 손실이 크게 진동할 가능성 존재 또는 발산 가능
- Rate가 너무 작은 경우 속도가 느리고, Epoch을 많이 주어야 함
- Step Decay (일정 epoch마다 rate 감소), ReduceLROnPlateau (검증 손실이 개선되지 않으면 rate 감소), Cosine Annealing (rate을 코사인 곡선 형태로 감소시킴) 등의 학습률 스케줄러가 존재

❖ Batch / Epoch

- Batch 경사하강법 : 전체 데이터를 한 번에 사용하여 기울기를 계산
- 확률적 경사하강법 : 한 개의 데이터마다 가중치를 업데이트
- Mini batch 경사하강법 : 여러 개의 데이터를 작은 묶음으로 나누어 학습
- Epoch이 너무 적은 경우 : 과소적합 발생 가능
- Epoch이 너무 많은 경우 : 과적합 발생 가능

❖ Optimizer

- Momentum
  - ✓ $v_t = \beta v_{t-1} + \nabla L(w_t)$
  - ✓ $w_{t+1} = w_t - \eta v_t$
  - ✓ 공이 경사면을 내려가며 관성을 얻는 것으로 비유 가능
- RMSprop
  - ✓ 각 가중치의 최근 기울기 크기를 이용해 학습률 조정
  - ✓ 기울기가 큰 방향은 적게, 기울기가 작은 방향은 크게 이동
- SGD
  - ✓ 구조가 단순
  - ✓ 학습이 느릴 수 있음
  - ✓ $w_{t+1} = w_t - \eta \nabla L(w_t)$
- ADAM (Momentum과 RMSprop의 장점을 결합)
  - ✓ 구조가 단순
  - ✓ 학습이 느릴 수 있음
  - ✓ $w_{t+1} = w_t - \eta \nabla L(w_t)$

Activation Function / Loss Function

Loss function

문제 유형	대표 Loss function
회귀 (Regression)	MSE, MAE, Huber
이진 분류	Binary Cross-Entropy
다중 클래스 분류	Categorical Cross-Entropy
다중 레이블 분류	Binary Cross-Entropy (레이블별)
임베딩·유사도 학습	Contrastive, Triplet Loss

Activation function

함수	주 사용 위치	특징 / 용도
ReLU	은닉층 (기본값)	계산 빠름, 기울기 소실 완화
Leaky ReLU / ELU	은닉층	죽은 뉴런 문제 보완
Sigmoid	이진 분류 출력층	출력을 0~1 확률로
Softmax	다중 분류 출력층	클래스별 확률 분포로
Tanh	은닉층 (RNN 등)	출력을 -1~1, 중심 0
Linear (없음)	회귀 출력층	값 범위 제한 없음

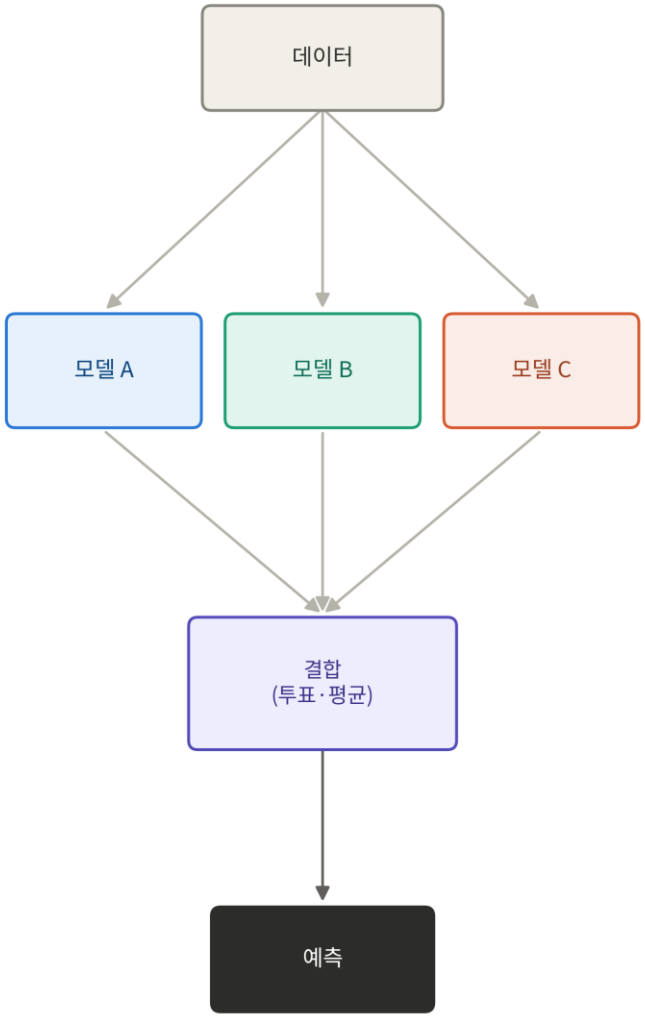
8.1.4 앙상블을 이용한 최적화

앙상블

- ❖ Ensemble은 여러 개의 모델을 함께 사용하여 하나의 최종 예측을 만드는 방법
- ❖ 앙상블은 개별 모델이 서로 다른 실수를 한다는 점에서 착안하여 하나의 모델이 틀리더라도 다른 모델이 이를 보완하는 방식
- ❖ 앙상블의 주요 효과
  - 정확도 향상 : 개별 모델보다 정확한 경우가 많음
  - 일반화 성능 향상 : 처음보는 데이터에서도 안정적일 가능성이 높음
  - 예측 분산 감소 : 모델마다 예측이 다르더라도 평균을 내면 결과의 흔들림 감소
  - 과적합 완화 : 하나의 모델이 패턴을 외우더라도 다른 모델이 이를 완화

앙상블이 효과적일 조건

- ❖ 앙상블을 적용할 때 모델들은 어느 정도 성능이 있으며, 실수가 서로 다른 방식이여야 함
  - 최소한의 무작위 보다는 좋은 성능을 가진 모델들
  - 데이터 샘플, 입력 값, 구조, 가중치, 활성화 함수, 옵티마이저 등이 다른 모델을 사용함



8.2 하드웨어를 이용한 최적화

CPU와 GPU의 차이

- ❖ CPU는 연산을 담당하는 ALU와 해석하고 실행하는 Control, 데이터를 담아두는 Cache로 구성
  - CPU는 ALU가 하나이므로 한번에 하나의 명령어만 처리
- ❖ GPU는 병렬처리 가능
  - Cache 비중은 낮고, ALU개수가 많아 짐으로써 병렬적으로 처리하도록 설계되었기 때문

CUDA

- ❖ GPU를 이용한 성능 최적화를 위해선 Pytorch의 CUDA가 필요
- ❖ CUDA는 NVIDIA에서 개발한 GPU 개발 툴
  - ❖ 이로 인해 전문가가 아니더라도 GPU를 이용한 프로그래밍을 쉽게 접근할 수 있게 됨

8.3 하이퍼 파라미터를 이용한 성능 최적화

배치 정규화를 이용한 성능 최적화

정규화(normalization)

- 데이터의 최솟값과 최댓값을 이용하여 범위를 일정하게 맞추는 방법
- $x_{norm} = \frac{x - x_{min}}{x_{max} - x_{min}}$
- 데이터를 0과 1사이로 변환
- 정규화의 장점
  - 경사하강법의 학습을 안정화
  - 학습 속도 향상 가능
  - 특정 변수가 지나치게 큰 영향을 미치는 현상 방지
- 정규화의 단점
  - 대부분의 값이 작으나, 하나의 이상치가 굉장히 크다면 대부분 데이터가 압축됨

표준화(standardization)

- 데이터의 평균을 0, 표준편차를 1에 가깝게 만드는 것
- $x_{std} = \frac{x - \mu}{\sigma}$
- 표준화의 장점
  - 데이터 단위와 크기의 차이를 줄임
  - 정규화보다 이상치의 영향을 덜 받음

전처리 주의사항

- 훈련 데이터에서만 계산해야 함
- 올바른 순서
  1. 전체 데이터를 train – test로 분리
  2. 훈련 데이터에서 정규화, 표준화 진행
  3. 계산된 값으로 훈련 데이터 변환
  4. 동일한 값을 사용하여 test 데이터 변환

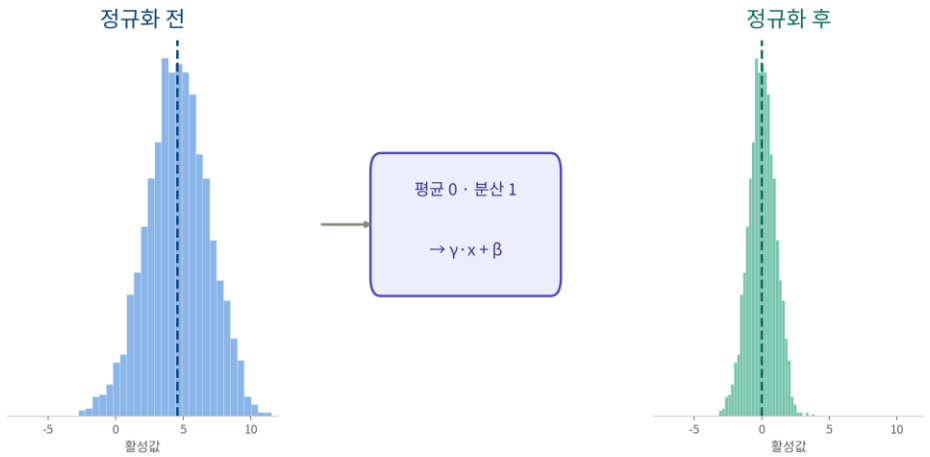
8.3 하이퍼 파라미터를 이용한 성능 최적화

Intelligent Design for Engineering Applications Lab

규제화(Regularization)

- ❖ Overfitting을 줄이기 위한 방법
- ❖ L1 정규화 (규제화), L2 정규화 (규제화) 존재
- ❖ Drop out : 학습할 때 일부 뉴런 무작위로 비활성화 하는 방법
- ❖ 조기 종료 (Early Stopping) : 검증 손실이 개선이 더 없을 경우 학습을 중단하는 방법

+ 데이터 증강도 훈련 데이터의 특정 형태만 외우는 것을 규제하는 점에서 규제화 효과를 가짐



Batch Normalization

- ❖ 신경망이 학습하는 동안 각 미니배치의 중간 출력 값을 평균 0, 분산 1에 가깝게 변환한 뒤, 학습 가능한 크기와 위치 조정 값을 적용
- ❖ 기울기 소멸과 폭발 원인으로 내부 공변량 변화 지적
  - 각 층마다 활성화 함수가 적용되며 입력 값의 분포가 계속 바뀜
  - 이러한 분포를 정규 분포로 만들기 위해 미니 배치에 표준화를 적용하는 방법 사용
  - 이를 배치 정규화라고 함
- ❖ 예시

1. 배치 평균 계산 : $\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i$
2. 배치 분산 계산 : $\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2$
3. 표준화 : $\hat{x_i} = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$
4. 크기와 위치 재조정 : $y_i = \gamma\hat{x_i} + \beta$ -〉 감마, 베타는 각각 크기, 위치의 학습가능 파라미터

- ❖ 모든 중간 값의 평균, 문산을 고정하면 다양한 분포 표현이 힘드므로, 표준화한 다음 모델이 필요한 분포를 스스로 결정하도록 두개의 파라미터를 둠

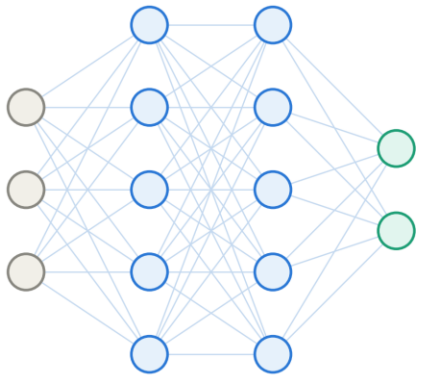
8.3.2 Drop out을 이용한 성능 최적화

Drop out 에 대해

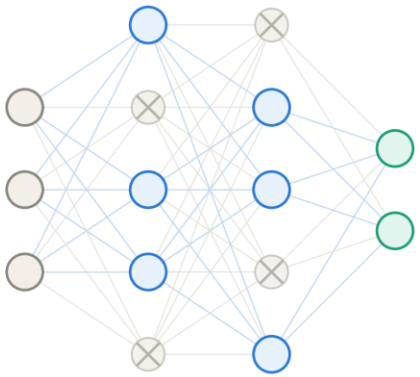
Drop out

- ❖ 신경망 학습 중 일부 뉴런을 무작위로 비활성화하는 규제화 기법
- ❖ 일반적인 ANN에서는 과적합 발생 가능 -〉 등장 이유

일반 네트워크



드롭아웃 적용



Drop out의 작동

- ❖ 작동 방식
  - 은닉층의 출력 : $h = [0.8, 0.5, 0.2, 0.9]$
  - Drop out 비율 : $p = 0.5$
  - 무작위 마스크 선택 : $m = [1, 0, 1, 0]$
  - 일부 출력 제거 : $h \odot m = [0.8, 0, 0.2, 0]$
  - 무작위 마스크는 다음 학습에서 다르게 선택될 수 있음

8.3.2 Drop out을 이용한 성능 최적화

Drop out 코드

```python
import torch
import torch.nn as nn


class ANN(nn.Module):
    def __init__(self, input_size: int):
        super().__init__()

        self.fc1 = nn.Linear(input_size, 64)
        self.relu1 = nn.ReLU()

        self.fc2 = nn.Linear(64, 32)
        self.relu2 = nn.ReLU()

        self.fc3 = nn.Linear(32, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.fc1(x)
        x = self.relu1(x)

        x = self.fc2(x)
        x = self.relu2(x)

        x = self.fc3(x)
        return x
```

```python
import torch
import torch.nn as nn


class ANNWithDropout(nn.Module):
    def __init__(self, input_size: int):
        super().__init__()

        self.fc1 = nn.Linear(input_size, 64)
        self.relu1 = nn.ReLU()
        self.dropout1 = nn.Dropout(p=0.3)

        self.fc2 = nn.Linear(64, 32)
        self.relu2 = nn.ReLU()
        self.dropout2 = nn.Dropout(p=0.3)

        self.fc3 = nn.Linear(32, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.fc1(x)
        x = self.relu1(x)
        x = self.dropout1(x)

        x = self.fc2(x)
        x = self.relu2(x)
        x = self.dropout2(x)

        x = self.fc3(x)
        return x
```

코드 설명

- 첫번째 은닉층에 drop out을 p=0.3 을 적용
- 64개의 노드 중에 30%의 뉴런의 출력을 0으로 만듦

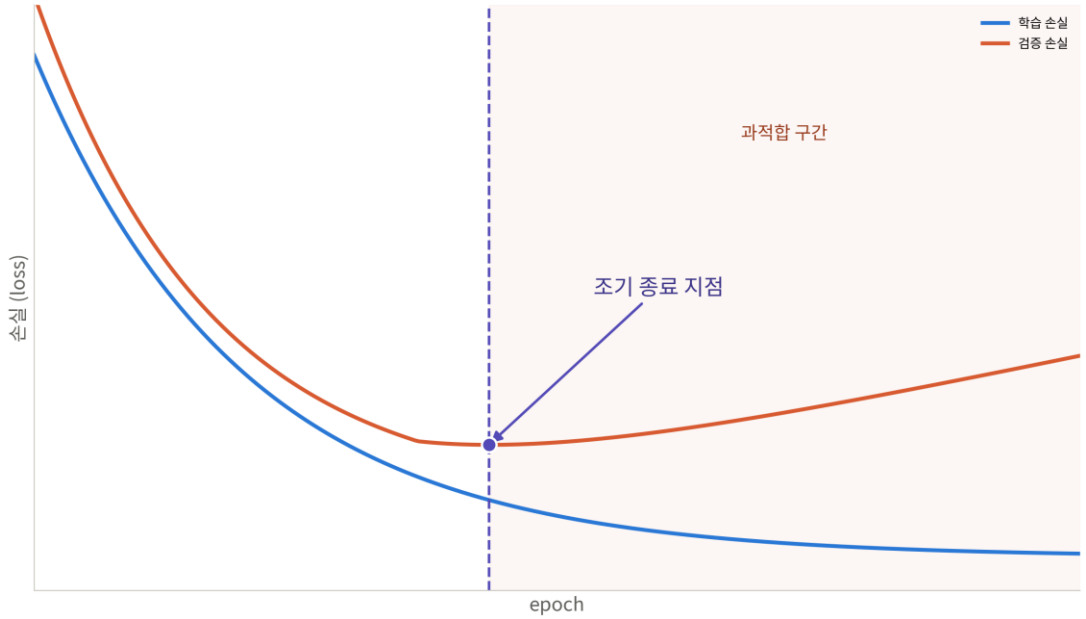
추가 설명

- batch norm을 같이 적용할 시 Linear 다음 batch norm -> ReLU -> Dropout 순서
- Drop out이 너무 큰 경우 손실이 잘 감소하지 않고 훈련 정확도가 떨어짐

8.3.3 조기 종료를 이용한 성능 최적화

Early stopping (조기종료)

- Neural Network가 과적합을 회피하는 규제 기법 중 하나
- 매 epoch마다 검증 데이터에 대한 오차를 측정하여 종료 시점 제어



조기 종료의 특징

- 일반적인 모델의 학습에서는 손실 감소 -> 적절한 학습 시점 -> 훈련 손실은 감소, 검증 손실은 증가 (과적합)의 형태
- 검증 손실을 관찰하며 patience코드로 n회까지 성능이 개선되지 않으면 학습을 중단하는 장치 존재
- min_delta로 인정할 최소 손실 변화량을 체크
- 일반적으로 검증 손실을 기준으로 보나, 분류 문제에서는 F1-score, 클래스 불균형이 심한 경우 Recall, AUROC의 지표가 더 적합할 수 있음

가장 좋은 모델 선정

- Patience을 설정해주고 epoch이 조기 종료되었을 때, 그중 가장 낮은 validation loss를 가진 epoch을 선정

8.3.3 조기 종료를 이용한 성능 최적화


```python
best_val_loss = float("inf")
patience = 5
counter = 0

for epoch in range(epochs):
    # 모델 학습
    train_model()

    # 검증 손실 계산
    val_loss = validate_model()

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        counter = 0
        torch.save(model.state_dict(), "best_model.pt")
    else:
        counter += 1

    if counter >= patience:
        print("Early stopping")
        break
```

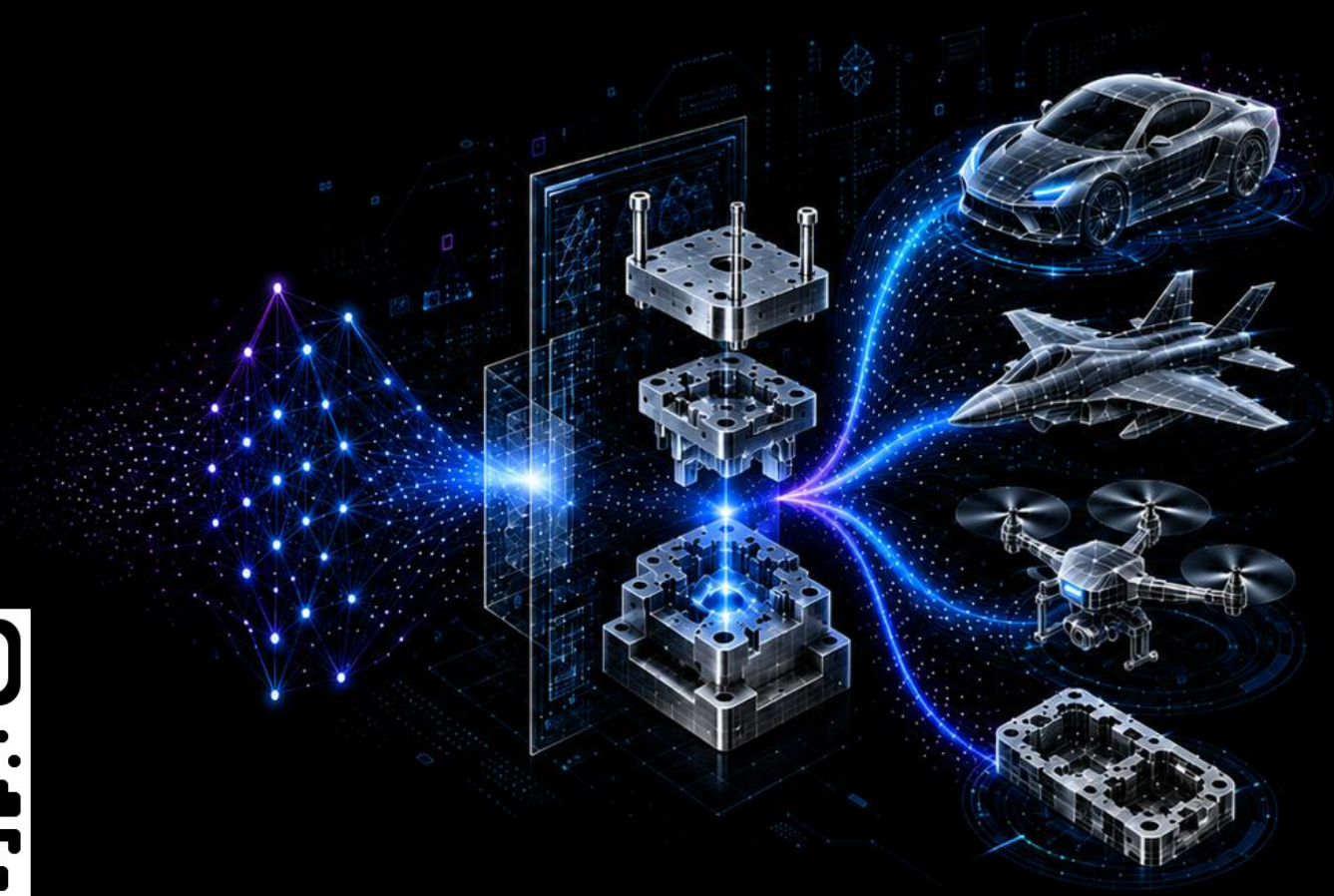
핵심 조건문 코드 설명

- best_val_loss=… : 현재까지 가장 낮은 검증 손실 저장
- patience=5 : 검증 손실이 좋아지지 않아도 5회 동안 종료 대기
- counter = : 검증 손실이 좋아지지 않은 횟수 세기

핵심 조건문 코드 설명

- if val_loss 〈 best_val_loss : 현재 검증 손실이 지금까지 최저 검증 손실보다 작은 지 확인 –〉 작아지면 참
- torch.save(model. : 손실이 작아지면 가중치 저장
- else : 성능이 개선되지 않으면 count +1
- if counter 〉= patience : 카운터가 patience보다 커지면 종료

Any Questions?



이름 정 승 한

소속 한양대학교 기계공학과

이메일 jshan0407@hanyang.ac.kr

Intelligent Design for Engineering Applications Lab